RoamPredict

How the code works, explained step by step

For the team: what the program does, when it switches access point and when it does not, what every log line means, and which file does what.

Code version described: commit fdfb6da (branch claude/upbeat-newton-gh5kgn) · 264 automated tests · Python 3.11+ standard library only · Windows 10/11

Contents

1. The problem and the idea in one page
2. How to run it, and the most common reason it seems to “do nothing”
3. The big picture: one loop, four times per second
4. Step by step through one tick
5. When RoamPredict switches, and why it usually does not
6. How a switch is done safely
7. Measuring the result: outages and the STOP line
8. Reading the log: every line explained
9. A real example, line by line
10. Which file does what (the three parts)
11. Lists and dictionaries in the code
12. Safety rules
13. Results so far and honest limits
14. Glossary

1. The problem and the idea in one page

When you walk across campus with a laptop, the Wi-Fi signal from your access point (AP) gets weaker. Windows decides by itself when to move to another AP. On our campus it waits too long: in our walks the **internet already stopped working 5 to 12 seconds before Windows gave up on the old AP**, and a Windows roam then took between 0.5 and 10 seconds.

RoamPredict watches the signal 4 times per second, predicts that the link is about to fail, and asks Windows to move to a better AP early, before the connection dies. It is not artificial intelligence: it uses filtering, a straight-line trend, a state machine and a scoring formula, all of which you can explain on a whiteboard.

Three questions run the whole program:

Question	How the code answers it	Part
Is the link about to fail?	Clean the readings, measure the slope, and let a state machine decide (STABLE, DEGRADING, ROAM_IMMINENT ...)	Part 1: Prediction
Which AP should we go to?	Score every visible AP of the same network; the winner must be clearly stronger (+8 dB)	Part 1: Prediction
Can we switch safely, and did it help?	Safety gate, then WlanConnect with the saved profile, then check the IP address; every outage is timed	Parts 2 and 3

Important words: **RSSI** = signal strength in dBm, always negative; -50 is excellent, -75 is weak, -85 is almost gone. **SSID** = the network name (“Campus Connecte”). **BSSID** = the hardware address of one AP (bc:a9:93:a2:22:00). One SSID has many BSSIDs.

2. How to run it

Open PowerShell in the project folder (the folder that contains `roampredict`, `README.md` and `pyproject.toml`). If `python` is not found, type `py` instead.

```
cd C:\Users\hp\roampredict-new
git pull                                    # get the newest code
python -m roampredict --version             # must say: code fdfb6da
python -m roampredict doctor                # health check, changes nothing
python -m roampredict run --probe www.google.com --duration 360            # WITHOUT acting
python -m roampredict run --active --probe www.google.com --duration 360   # WITH acting
python -m roampredict report --days 0.1 --timeline 150                     # what happened
```

Part of the command	Meaning
run	Start the watching loop.
--active	Allow real switches. **Without it the program only watches** (dry-run) and writes WOULD_SWITCH instead of switching.
--probe www.google.com	Also check the internet: a TCP connection to www.google.com port 443 every 0.25 s. This is what measures internet outages.
--duration 360	Stop by itself after 360 seconds (6 minutes).
report --timeline	List the outages and roams saved in the database, in time order.

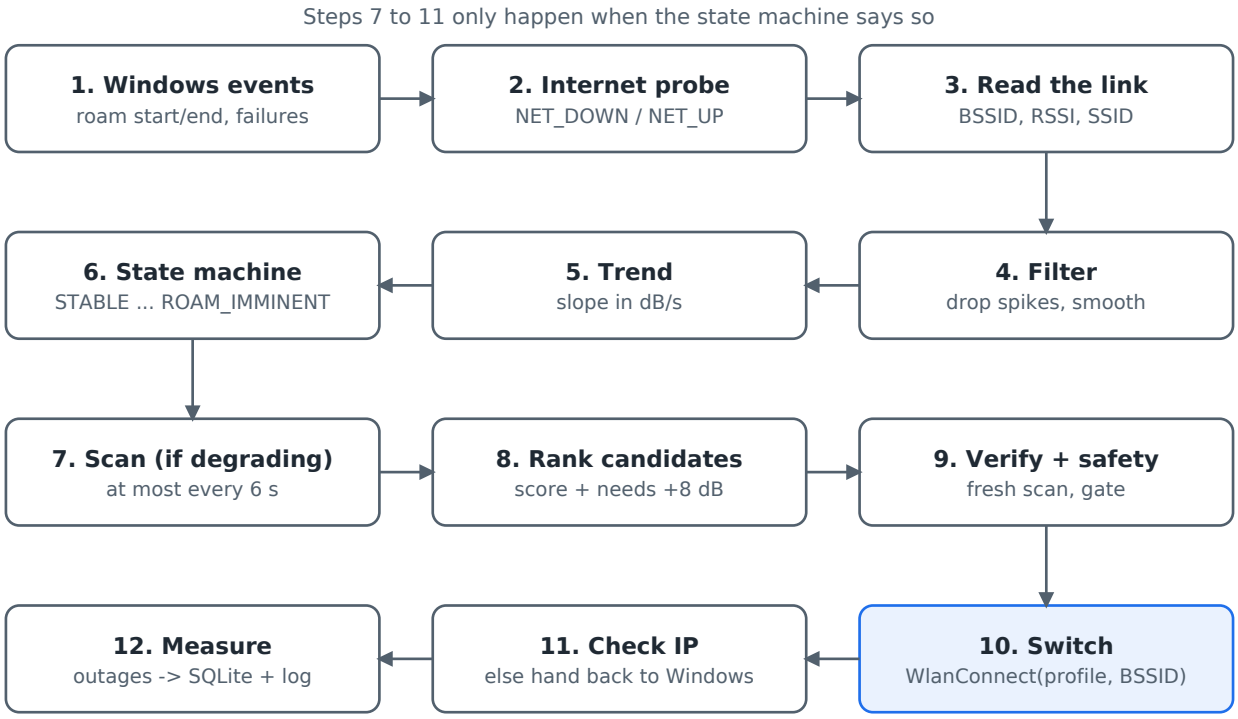
“It doesn’t work”: check these first

What you see	What it means	What to do
START line says DRY-RUN	You forgot --active. It will never switch.	Ctrl+C, run again with --active.
“Refusing active mode: no SSIDs are allow-listed”	The config file has no allow-list.	Create config.toml with [safety] allowed_ssids = ["Campus Connecte"].
“Refusing active mode: the kill switch is active”	It stopped itself earlier (2 failed switches or a possible login prompt).	Run: python -m roampredict unkill
Only STATE, NO_CANDIDATE, WINDOWS_ROAM lines	It is working. No AP was good enough, or Windows roamed first.	Normal. See section 5.
“No module named roampredict”	PowerShell is not in the project folder.	cd into the folder first.
“unrecognized arguments: --probe”	Old copy of the code.	git pull, check --version.

Key point: RoamPredict only acts in a short window: when the link is predicted to fail (state ROAM_IMMINENT) **and** another AP of the same network is clearly better **and** every safety rule allows it. Most of the time none of this is true, so it correctly does nothing. A walk with zero switches is not a crash.

3. The big picture: one loop, four times per second

The program is one loop (`Engine.run` in `engine.py`). Every 0.25 s it calls `Engine.tick`, which does the steps below in this order. The internet probe runs at the same time in its own thread.



Steps 1 to 6 happen on every tick. Step 7 happens only in DEGRADING or ROAM_IMMINENT. Steps 8 to 11 happen only in ROAM_IMMINENT. In dry-run, step 10 is replaced by the line WOULD_SWITCH.

Who calls whom

Object (file)	Job	Used by
Engine (engine.py)	The loop, all decisions, all outage timing	cli.py (command run)
WindowsWlanAdapter (platform/windows_wlan.py)	Talks to Windows' Wi-Fi library wlanapi.dll through ctypes	Engine, RoamExecutor
SignalFilter, SlidingRegression (signal/)	Clean readings, compute the slope	Engine
RoamStateMachine (state_machine.py)	Decides the state	Engine
Predictor (topology/predictor.py)	Scores the candidate APs	Engine
TopologyStore (topology/store.py)	SQLite memory: APs, moves, switches, outages	Engine, Predictor, report
SafetyGate (safety.py)	Dry-run, allow-list, kill switch, rate limit	Engine
RoamExecutor (roam/executor.py)	Does one safe switch and measures it	Engine
ConnectivityProbe (net/probe.py)	Internet check in a background thread	Engine
Telemetry (telemetry.py)	Screen lines and the JSON log file	everyone

4. Step by step through one tick

Steps 1-3: what is happening right now

Windows events: Windows sends notifications (connection started, roaming started, disconnected, failure with a reason code). A callback stores them; the engine reads them at the start of each tick. This is how it notices that **Windows roamed on its own** (line WINDOWS_ROAM) or that a connection failed (line WARNING with the reason, e.g. 0x38002 “driver disconnected while associating”).

Internet probe: every 0.25 s it opens a TCP connection to www.google.com:443 (the name is looked up only once). **Two unanswered probes in a row = an internet outage** (NET_DOWN). The first answer ends it (NET_UP, with the duration). One lost probe is ignored, because single packets get lost on any network.

Read the link: `WlanQueryInterface` gives the current BSSID, SSID, profile name and signal. On the Intel AX201 of our main laptop the driver returns nonsense for dBm (+27), so the code uses signal quality instead: `dBm = quality / 2 - 100`. If the link is down, the engine starts a Wi-Fi outage timer and waits for Windows to reconnect.

Step 4: the filter (signal/filter.py)

Raw readings jump around: a hand over the antenna can drop the signal 10 dB for half a second. Two stages:

- **Spike rejector**: a reading more than 6 dB away from the median of the last 5 readings is held back. It is accepted only if 2 readings in a row agree on the new level. So one strange reading never causes a roam.
- **Smoother (EMA)**: an exponential moving average with a 1-second time constant: `smoothed = smoothed + a * (reading - smoothed)`, where a depends on the time between readings. It removes noise but adds a small delay.

Step 5: the trend (signal/trend.py)

The code fits a straight line (least squares) through the smoothed readings of the **last 3 seconds**. The slope of the line is the speed of change in dB per second. It is trusted only with at least 6 readings and a clear trend (the slope is at least 2 times its own uncertainty). From the slope it computes the **time to threshold**: how many seconds until -75 dBm at this speed.

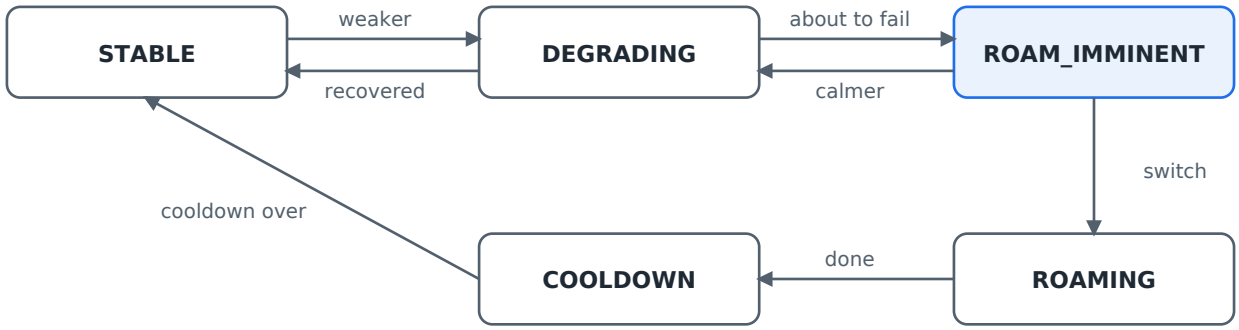
> Example: signal -70 dBm, falling 3 dB/s: (-70 - (-75)) / 3 = 1.7 seconds left. Signal -70 dBm but steady: “never”. Same level, completely different danger: that is why the slope matters.

Step 6: the state machine (state_machine.py)

The state machine turns level and slope into one of five states. Thresholds:

Rule	Value
Enter DEGRADING	signal below -67 dBm, or falling and -75 dBm predicted within 10 s
Back to STABLE	signal above -63 dBm and not falling (hysteresis: -67 to enter, -63 to leave)
Enter ROAM_IMMINENT	signal at or below -75 dBm, or falling at least 1 dB/s and -75 dBm predicted within 4 s, or (with --probe) the internet stopped answering while the signal is -65 dBm or weaker
Hold times	a worse condition must last 0.5 s, a better one 2 s (stops flickering)
COOLDOWN	20 s after our switch, 60 s after a failed one, 5 s after Windows roamed by itself

ROAM_IMMINENT is the only state where RoamPredict may switch



STABLE can also jump straight to ROAM_IMMINENT when the signal collapses fast. When Windows roams by itself, any state goes to COOLDOWN for 5 s, then the readings start again from zero (for 9 s the slope is ignored on laptops that report quality, because the first reading after a reconnect is a fake -50).

5. When RoamPredict switches, and why it usually does not

A switch happens only if **all** of these are true, checked in this order. Each “no” has its own log line, so you can always see why nothing happened.

#	Condition	If not: what you see
1	State is ROAM_IMMINENT	only STATE lines
2	Not in COOLDOWN (20 s after a switch, 60 s after a failure, 5 s after Windows roamed)	nothing new

#	Condition	If not: what you see
3	Another AP of the **same SSID** is visible, heard in the last 30 s	NO_CANDIDATE … last heard N s ago
4	It is at least **8 dB stronger** than the current AP (2.4 GHz APs count 7 dB weaker)	NO_CANDIDATE … only +5.4 dB vs current (needs +8)
5	It is not an AP we left less than 45 s ago (unless +12 dB)	… ping-pong guard
6	If the usual next AP is visible and the link still has time: wait for it	WAITING
7	A fresh scan (max 3 s old) confirms it, and the link did not change meanwhile	NOT_VERIFIED
8	Safety gate: --active, kill switch off, SSID allow-listed, saved profile, rate limit	WOULD_SWITCH … not acting: dry-run

How the candidates are scored (topology/predictor.py)

Every candidate gets a score between 0 and 1. The score orders the candidates; the +8 dB rule decides whether they are allowed at all.

Part of the score	Weight	Meaning
learned	0.35	How often you went from this AP to that one before (counts stored in SQLite)
signal	0.35	Signal strength scaled from -90 dBm (0) to -45 dBm (1)
seen recently	0.10	Heard in the last scan, fading over about 20 s
connected recently	0.20	You were on it in the last 12 hours (its key may still be cached: faster login)

> Worked example: you are at -72 dBm. The only candidate is at -60 dBm on 5 GHz, heard 2 s ago, never used. Gain = -60 - (-72) = +12 dB, more than 8, so it is allowed. Score = 0.35 × 1 (learned, it is the only one) + 0.35 × 0.67 (signal) + 0.10 × 0.90 (seen) + 0.20 × 0 = **about 0.67**.

The dead-link rule (added after the field tests)

When the internet is down, the signal reading of our laptop often **freezes** at its last value, so the current AP looks fine when it is dead. With --probe, if the internet stopped answering and the signal is -65 dBm or weaker, the state becomes ROAM_IMMINENT. The current AP is then judged by what the **last scan** measured for it (or -85 dBm if the scan did not hear it at all), and the **strongest** AP that beats it by **3 dB** is taken. You see this in the log as “internet down; the last scan measured the current AP at -80 dBm; taking the strongest AP (needs +3 dB)”.

6. How a switch is done safely (roam/executor.py)

Windows has no public “roam now” command. The only tool is `WlanConnect`: “connect with this saved profile, and I would like this BSSID”. Windows usually drops the old link and connects again, so a switch always costs a short break (2 to 7 s measured). That is why the rules in section 5 are strict.

1. **Verify**: re-read the AP list; if the candidate was not heard in the last 3 s, ask for a scan and wait (max 4.5 s). Check it is still there and still strong enough.
2. **Connect**: `WlanConnect(profile name, one desired BSSID)`. Only the **name** of the saved profile is passed. The password or certificate stays inside Windows; Python never sees it.
3. **If the connection fails**: if 802.1X (the login step) stalls or fails, a login window might appear, so it stops, reports it and trips the kill switch. Otherwise it gives Windows 5 s to reconnect by itself, then reconnects by profile name only (Windows chooses the AP).
4. **Check the BSSID**: did Windows really connect to the AP we asked for? If not, it is reported.
5. **Check the IP layer**: wait up to 4 s for a real address and gateway. Same network: flush the ARP cache; new network: renew DHCP (both need administrator rights, otherwise skipped). Our campus blocks ping, so the address is checked instead of pinging the gateway.
6. **No working IP address** (seen on Aymane's laptop: DHCP never answered, Windows gave itself 169.254.x): **new in fdfb6da**, it reconnects by profile name only, so Windows chooses the AP again and the laptop is not left offline on the AP we asked for.
7. **Measure and remember**: connect time, login (auth) time, IP time, success or failure go to SQLite. Then COOLDOWN starts (20 s, or 60 s after a failure).

A switch is “recent”, “expired” or “never” depending on whether you were connected to that AP in the last 12 hours. The report compares their switch times: if “recent” switches are clearly faster, the campus caches login keys (PMKSA caching).

7. Measuring the result: outages and the STOP line

Two kinds of outage are timed and saved in the SQLite table `outages`, with the mode (dry-run or active):

- **Wi-Fi outage**: from the moment the link was lost until a new connection (LINK_DOWN ... CONNECTED).
- **Internet outage**: from the first unanswered probe until the first answer (NET_DOWN ... NET_UP). This is what users feel, so it is the main result.

During the run they are also collected in a dictionary of lists, `session_outages = {"wifi": [], "internet": []}`. At the end, `session_summary()` counts them, adds them up and takes the longest:

```
STOP  RoamPredict stopped (0/0 switches succeeded). This session: Wi-Fi link lost 5x,
      48.5 s in total (longest 21.9 s); internet lost 12x, 162.1 s in total (longest 34.0 s);
      Windows roamed 10x
```

Our own switches count as outages too, measured from the moment we asked, so RoamPredict can only look worse, never better, than Windows alone. **Fair comparison**: same laptop, same route, same length, run once without --active and once with, and compare “internet lost” as a percentage of the walk.

8. Reading the log: every line explained

Line	Meaning
START	Program started; says DRY-RUN or ACTIVE and which code version (commit) runs.
CONNECTED	Connected to an AP (at start, or after a loss: “after 14.5 s without Wi-Fi”).
STATE	The state machine changed state, with the reason (rssi, slope, time-to-threshold).
NO_CANDIDATE	Wanted to move, but no AP qualifies; names the closest one and why it failed.
WAITING	Waiting for the usual next AP, because the link still has a few seconds.
NOT_VERIFIED	The best candidate failed the fresh check (not heard recently, or not strong enough now).
WOULD_SWITCH	Dry-run: everything allowed a switch, but it did not act.
DECISION	Active: switching now, with the gain, score and reason.
SWITCH_RESULT	OK or FAILED, with connect time, auth (login) time, IP time and the fallback used.
WINDOWS_ROAM	Windows moved to another AP by itself, with how long it took.
PREDICTION_CHECK	Dry-run: did Windows pick the AP we would have picked? HIT or MISS, and how much earlier we decided.
LINK_DOWN	Wi-Fi link lost, with the last readings before the loss.
NET_DOWN / NET_UP	Internet stopped answering / answers again, with the duration and lost probes.
IP_CHANGE	IP address changed; 169.254.x means DHCP did not answer, so no internet.
WARNING	Windows reported a failed connection attempt, with its reason code and text.
KILL_SWITCH	The emergency stop is on or off.
STOP	End of the session with the totals (section 7).

Windows reason codes we met on campus

Code	Text	What it means
0x38002	The driver disconnected while associating	The AP did not accept the connection (weak signal, busy AP)
0x38001	The operation was cancelled	A connection attempt was interrupted
0x2800b	The specific network is not available	No AP of that network heard at that moment
0x50006	The authenticator is no longer present	The AP disappeared during the login step
0x4800F	No authenticator, but the profile requires 802.1X	The login step never started

9. A real example, line by line

From Aymane's active walk (his laptop clock), slightly shortened:

```
16:44:03 NET_DOWN       no answer from www.google.com:443 since 16:44:03.4
16:44:06 STATE          DEGRADING -> ROAM_IMMINENT: internet stopped answering at -68.9 dBm
16:44:10 NOT_VERIFIED   not switching to ...a0:30: no fresh data for the candidate (last heard: 24.1 s)
16:44:14 DECISION       switching ...9a:f0 -> ...24:30 (+27.0 dB, score 0.32, never; internet down
                        and the last scan did not hear the current AP: counted as -85 dBm;
                        taking the strongest AP (needs +3 dB))
16:44:16 SWITCH_RESULT  OK: switched [never, connect 1914 ms (auth 773 ms), IP ready after 13 ms]
16:44:22 NET_UP         internet answers again after 19.0 s
```

1. The probe misses 2 answers in a row: internet outage starts (NET_DOWN).
2. The signal is -68.9 dBm (weaker than -65) and the internet is dead: the dead-link rule makes the state ROAM_IMMINENT.
3. The first candidate was last heard 24 s ago: too old, so it is refused (NOT_VERIFIED). A fresh scan runs.
4. The scan no longer hears the current AP, so it counts as -85 dBm. The strongest AP is 27 dB better, far more than the 3 dB needed: DECISION.
5. WlanConnect takes 1.9 s, of which 0.8 s is the 802.1X login; the IP address is ready 13 ms later.
6. 6 s after the switch the internet answers again. Total internet outage: 19 s.

Same walk, 22 s later, the third switch connected in 1.6 s but DHCP never gave an address (IP_CHANGE to 169.254.x). The old code stayed on that AP. Since fdfb6da it hands the connection back to Windows. This is how the field tests improve the code.

10. Which file does what (the three parts)

Part	Files to master	Files to skim
1. Prediction: when to move, and where to	signal/filter.py (216 lines), signal/trend.py (158), state_machine.py (252), topology/predictor.py (260)	topology/store.py (570), simulator/ (1,559)
2. Windows and network: see, scan, connect, check the internet	platform/base.py (332), platform/windows_wlan.py (649), net/probe.py (177), validate.py (114)	platform/info_elements.py, net/l3.py, net/latency.py, doctor.py
3. Engine, safety, measuring: the loop, the rules, the results	engine.py (936), safety.py (167), roam/executor.py (463)	cli.py, config.py, telemetry.py, buildinfo.py

Every path is inside the `roampredict` folder except `simulator/`. The simulator (`python -m roampredict simulate --no-plot --seeds 3`, 2 seconds, any laptop) compares Windows alone, a simple threshold and RoamPredict on a virtual school floor. The tests: `py -m pip install pytest` once, then `py -m pytest`.

11. Lists and dictionaries in the code

Both are used everywhere. Each presenter can show the examples of their part and say why that structure fits.

Part	Where	Code	Why this structure
1	topology/predictor.py: 180	pool: dict[str, CandidateInput] = {}	Dictionary keyed by BSSID: one entry per AP, a duplicate keeps the stronger reading

Part	Where	Code	Why this structure
1	topology/predictor.py: 194, :249	results.append(...); results.sort(key=...)	List: keeps an order, eligible and best-scored first
1	topology/store.py:484	groups.setdefault(key, []).append(rec)	Dictionary of lists: groups switch records to compute median times
2	platform/windows_wlan.py:194	PROTOTYPES: dict[str, tuple[list, object]]	Dictionary as a table: 11 Windows functions set up by one loop
2	platform/windows_wlan.py:73	_ACM_KINDS = {...}	Lookup table: Windows event number -> event name, no long if/elif
2	doctor.py:80	[{k: str(v) for k, v in item.items()} for item in items]	List of dictionaries from a list comprehension around a dict comprehension
3	engine.py:200	session_outages = {"wifi": [], "internet": []}	Dictionary of lists: one list of durations per kind of outage
3	config.py:85	SCHEMA: dict[str, dict[str, type]]	Nested dictionary: section -> key -> type; unknown keys are rejected
3	cli.py:61, :83	events: list[tuple[float, str]]; events.sort(); events[-last:]	List of tuples: sort by time, keep the last N

Other structures you can name: tuples (fixed pairs like (time, dBm)), deque (sliding window of readings, signal/trend.py:101), frozenset (allowed values, topology/store.py:120), Counter (why probes were lost, net/latency.py:114), dataclasses (records such as LinkStatus), Enum (the five states).

12. Safety rules

- **Dry-run by default.** Nothing is switched without --active.
- **Own device, own allow-listed networks only.** Active mode refuses to start without an allow-list, and acts only on SSIDs listed in config.toml.
- **No passwords.** Only the saved profile's name is passed to Windows; the log removes anything that looks like a credential.
- **No login prompts.** If the 802.1X login step stalls or fails, it stops, reports it and leaves everything to Windows (the kill switch trips).
- **Never disconnect without a verified better AP** (fresh scan, +8 dB, or +3 dB on a dead link).
- **Rate limit**: at least 20 s between switches, at most 6 per 10 minutes; stops after 2 failed switches in a row.
- **Kill switch**: `python -m roampredict kill` from another window stops all switching; `unkill` allows it again. Ctrl+C stops the program.
- **Every BSSID is validated** before it reaches Windows. No deauthentication, spoofing or packet injection, and no Linux-only tools.

13. Results so far and honest limits

Walk (main laptop unless noted)	Mode	Internet lost	Note
Atelier, 6 Oct, 7:24	without	230 s (52%)	11 outages, hops to RobotiCore
Atelier, 6 Oct, 5:41	with	136 s (40%)	2 switches: 1 OK (internet back 6.6 s after), 1 failed
7 Oct, walk 1, 6:00	without	331 s (92%)	One 257 s outage: Windows' login kept failing
7 Oct, walk 2, 6:00	without	162 s (45%)	10 Windows roams, 2 hops to RobotiCore
7 Oct, Aymane, 6:00	without	114 s (32%)	His laptop roams 11 times by itself
7 Oct, Aymane, first 2 min	with	93 s of 129 s (72%)	3 switches: 2 OK, 1 left without IP; battery died

What we can honestly say: Windows alone, on the same laptop and route, varied from 45% to 92% of the walk offline. The individual outages that RoamPredict handled were short (6.6 s and 19 s against a median of about 36 s for Windows on the first day), but there are too few active walks to claim a general improvement. The field tests found and fixed real problems: a driver giving nonsense dBm, a signal reading that freezes when the link is dead, a crash on large error codes, and a switch that left the laptop without an IP address.

Limits

- Windows has no “roam now” command; WlanConnect drops the link first, so every switch costs a few seconds.
- After a switch by BSSID, some drivers may keep the laptop on that AP (“pinning”); `netsh wlan connect name="Campus Connecte"` releases it.
- RoamPredict cannot help while the laptop is disconnected or when Windows' login (802.1X) keeps failing.
- Two laptops with the same Wi-Fi card behaved very differently, so results must be compared on the same laptop.
- Few walks: the numbers are examples, not proof. Windows only.

14. Glossary

Word	Meaning
AP / BSSID	Access point / its hardware address (one AP, one BSSID per band)
SSID	Network name, e.g. Campus Connecte
RSSI, dBm	Signal strength; closer to 0 is stronger (-50 good, -75 weak, -85 almost gone)
Roam	Moving from one AP to another of the same network
Dry-run	Watch and log only, never switch
802.1X, WPA2-Enterprise	Login with a university account; Windows does it with the saved profile
DHCP, 169.254.x	How a laptop gets its IP address / the address Windows invents when DHCP does not answer
Probe	The internet check: TCP connection to www.google.com:443 every 0.25 s
EMA	Exponential moving average, the smoother
Slope	Speed of signal change in dB per second, from a straight line through the last 3 s
Hysteresis	Different thresholds to enter and to leave a state (-67 / -63), so it does not flicker
ctypes	Python's built-in way to call Windows DLL functions such as wlanapi.dll
SQLite	The small database file that stores APs, moves, switches and outages
Kill switch	A file that, while it exists, blocks every switch
802.11r/k/v	Fast-roaming features the campus APs advertise; Windows uses them in its own roams